PQ

PulseQueue

A Production-Inspired Distributed Job Scheduler Platform

Student Name: Chavan Rupesh Sharan

Registration Number: RA2311026010210

Submission Date: July 04, 2026

GitHub Repository:
https://github.com/RupeshSharan/Codity.ai_assigment.git

Intern Assignment Submission - Production Engineering Track

PulseQueue

PulseQueue is a production-inspired distributed job scheduling platform for asynchronous background work. It is built around project isolation, configurable queues, atomic job claiming, worker heartbeats, retries, execution history, and a dashboard-oriented operations experience.

Assignment Fit

The original assignment emphasizes engineering quality over feature count. This implementation therefore focuses on the highest-scoring areas first:

- Relational schema for users, organizations, projects, queues, jobs, executions, retry policies, workers, heartbeats, logs, and Dead Letter Queue entries.
- FastAPI REST APIs with validation, authentication, structured errors, pagination-ready job listing, and operational routes.
- Atomic claim service that prevents duplicate execution and respects paused queues plus queue concurrency limits.
- Worker runner, scheduler tick, heartbeat recovery, retries, and DLQ transitions.
- React dashboard scaffold for overview metrics, queues, jobs, workers, and failed-job operations.
- Docker Compose for PostgreSQL, Redis, API, scheduler, worker, and frontend.
- Pytest coverage for critical reliability behavior.

Stack

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS, Recharts
Backend	FastAPI
Database	PostgreSQL in Docker, SQLite fallback for local dev/tests
ORM/Migrations	SQLAlchemy, Alembic
Cache	Redis
Auth	Signed JWT, PBKDF2 password hashing
Tests	Pytest
Runtime	Docker Compose

Quick Start

Copy the environment template:

```
cp .env.example .env
```

Run the full stack:

```
docker compose up --build
```

Open:

- API health: `http://localhost:8000/health`
- API docs: `http://localhost:8000/docs`
- Dashboard: `http://localhost:5173`

Local Backend Development

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload
```

Run tests:

```
cd backend
python -m pytest -q
```

Worker and Scheduler

Run a scheduler loop:

```
cd backend
python -m app.scheduler.service
```

Run a worker:

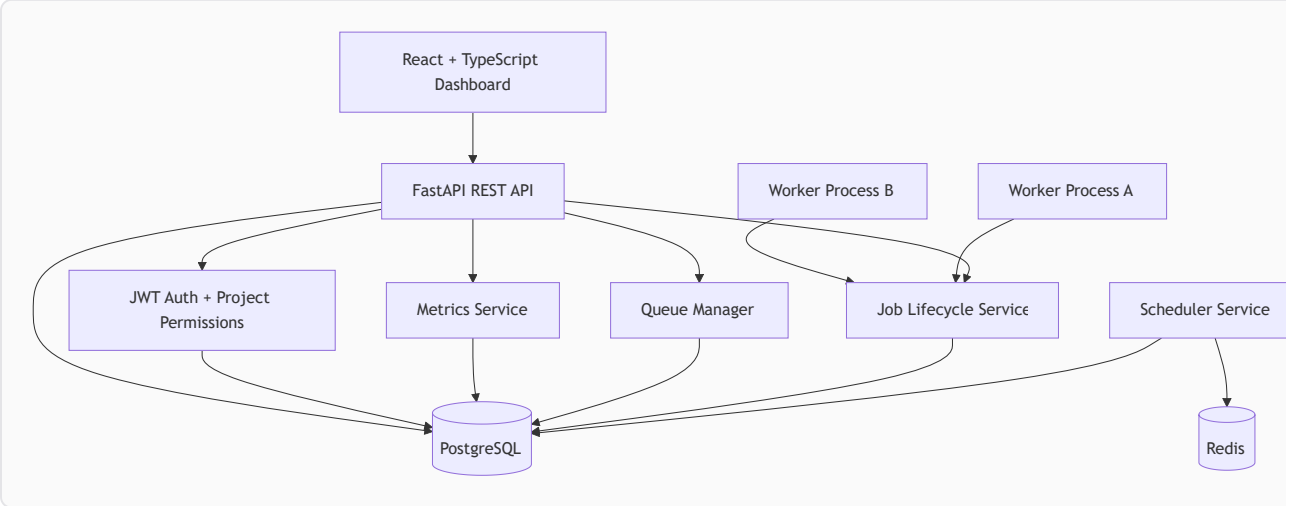
```
cd backend
set WORKER_ID=worker-local-1
set WORKER_CONCURRENCY=4
python -m app.workers.runner
```

Key Documents

- Assignment Comparison
- Architecture
- ER Diagram
- API Documentation
- Database Design
- Design Decisions
- Next Steps

Architecture

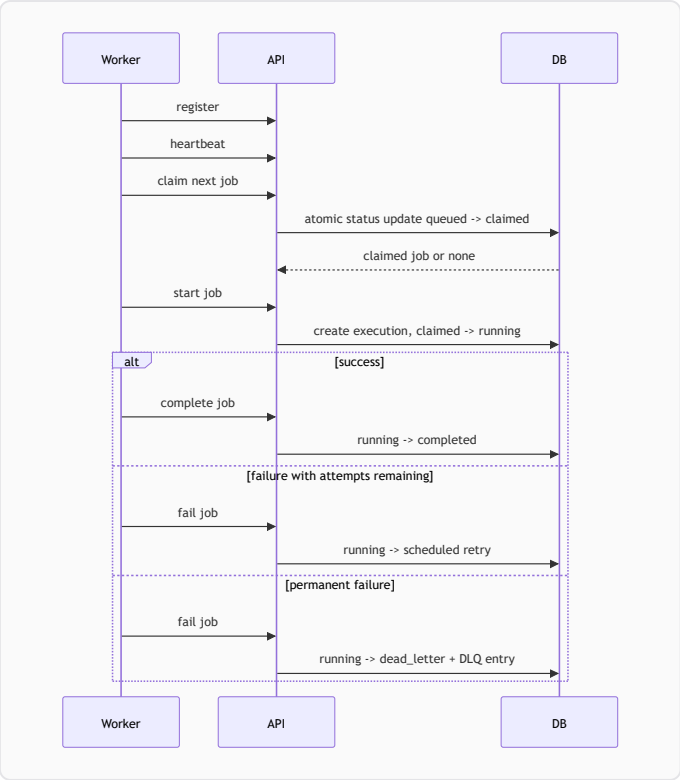
PulseQueue separates control-plane APIs from background execution. Users and operators interact through the React dashboard and REST API. Workers are independent processes that poll for work, claim jobs atomically, execute payload handlers, and report lifecycle transitions. A scheduler service handles delayed jobs and stale-worker recovery.



Runtime Services

- `api`: FastAPI app exposing `/api/v1`.
- `scheduler`: Promotes due scheduled jobs, marks stale workers offline, and requeues stale claimed/running jobs.
- `worker`: Registers itself, sends heartbeats, claims jobs, executes jobs concurrently, and reports success/failure.
- `postgres`: Primary relational store.
- `redis`: Reserved for cache/rate limiting/distributed locking extensions.
- `frontend`: React dashboard.

Reliability Flow



Atomic Claiming

The `claim_next_job` service scans eligible jobs by priority and scheduled time, filters out paused queues, checks queue concurrency, and performs a guarded status update. PostgreSQL can use row locks with `SKIP LOCKED`; SQLite tests use a guarded update to prove only one caller transitions a queued job to claimed.

Database Design

Core Tables

- `users`: Auth identities with unique email and PBKDF2 password hashes.
- `organizations`: Top-level ownership boundary.
- `projects`: Isolation boundary for queues, jobs, retry policies, and membership.
- `project_members`: Role mapping between users and projects.
- `retry_policies`: Fixed, linear, or exponential retry configuration.
- `queues`: Processing lanes with priority, max concurrency, retry policy, and pause/resume status.
- `jobs`: Main lifecycle table with payload, metadata, priority, schedule, attempts, locks, and timestamps.
- `job_executions`: Attempt history for every run.
- `job_logs`: Structured audit trail for creation, claim, start, completion, retry, and DLQ transitions.
- `workers`: Worker registry and latest heartbeat.
- `worker_heartbeats`: Historical heartbeat samples for observability.
- `dead_letter_entries`: Permanent failure records with payload snapshot and final error.

Important Indexes

- `users.email`: Fast login lookup.
- `organizations.slug`: Stable URL-safe organization identity.
- `projects.organization_id + slug`: Organization-scoped project lookup.
- `queues.project_id + status + priority`: Queue listing and scheduler scans.
- `jobs.status + scheduled_at + priority + created_at`: Claim scan ordering.
- `jobs.queue_id + status`: Queue health and backlog metrics.
- `jobs.project_id + idempotency_key`: Idempotent job creation.
- `worker_heartbeats.worker_id + recorded_at`: Worker monitoring.
- `job_logs.job_id + created_at`: Job timeline.

Cascading Behavior

- Deleting an organization deletes its projects.
- Deleting a project deletes queues, retry policies, memberships, and jobs.
- Deleting a queue deletes its jobs.
- Deleting a job deletes executions and logs.
- Worker references on jobs/executions are set to null where historical preservation is preferred.

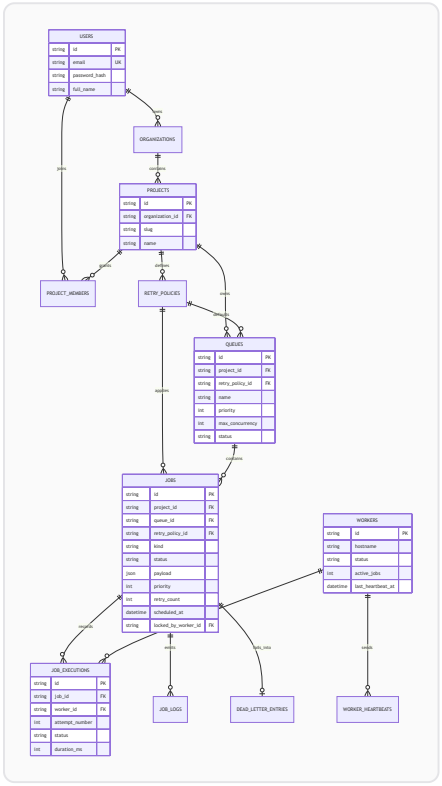
Performance Considerations

The claim query is designed for a high-write table:

1. Filter by status and scheduled time.
2. Exclude paused queues.
3. Respect queue concurrency by counting active jobs for the queue.
4. Order by job priority, queue priority, scheduled time, and creation time.
5. Transition `queued -> claimed` with a guarded update.

For larger deployments, the next optimization would be partitioning `jobs` by project or queue, adding a partial index for claimable jobs, and moving high-volume logs to a time-series or append-optimized store.

ER Diagram


USERS
string id PK
string email UK
string password_hash
string full_name
owns
ORGANIZATIONS
belongs
PROJECTS
string id PK
string organization_id FK
string slug
string name
joins
invites
PROJECT_MEMBERS
configures
RETRY_POLICIES
creates
defines
QUEUES
string id PK
string project_id FK
string retry_policy_id FK
string name
int priority
int max_concurrency
string status
contains
submits
JOBS
string id PK
string project_id FK
string queue_id FK
string retry_policy_id FK
string kind
string status
json payload
int priority
int retry_count
datetime scheduled_at
string locked_by_worker_id FK
WORKERS
string id PK
string hostname
string status
int active_jobs
datetime last_heartbeat_at
records
JOB_EXECUTIONS
string id PK
string job_id FK
string worker_id FK
int attempt_number
string status
int duration_ms
emits
JOB_LOGS
fails_into
DEAD_LETTER_ENTRIES
sends
WORKER_HEARTBEATS

API Documentation

FastAPI generates interactive OpenAPI docs at `/docs`.

Authentication

Method	Path	Purpose
`POST`	`/api/v1/auth/register`	Create a user and first organization
`POST`	`/api/v1/auth/login`	Exchange email/password for bearer token
`GET`	`/api/v1/auth/me`	Read current user

Organizations and Projects

Method	Path	Purpose
`GET`	`/api/v1/organizations`	List organizations visible to the user
`POST`	`/api/v1/organizations`	Create organization
`POST`	`/api/v1/organizations/{organization_id}/projects`	Create project
`GET`	`/api/v1/organizations/{organization_id}/projects`	List projects

Queues and Retry Policies

Method	Path	Purpose
`POST`	`/api/v1/queues/retry-policies/{project_id}`	Create retry policy
`GET`	`/api/v1/queues/retry-policies/{project_id}`	List retry policies
`POST`	`/api/v1/queues`	Create queue
`GET`	`/api/v1/queues?project_id=...`	List queues
`PATCH`	`/api/v1/queues/{queue_id}`	Update queue config
`POST`	`/api/v1/queues/{queue_id}/pause`	Pause queue
`POST`	`/api/v1/queues/{queue_id}/resume`	Resume queue
`GET`	`/api/v1/queues/{queue_id}/stats`	Queue statistics

Jobs

Method	Path	Purpose
`POST`	`/api/v1/jobs`	Create immediate, delayed, scheduled, recurring, or tagged job
`POST`	`/api/v1/jobs/batch`	Submit batch jobs
`GET`	`/api/v1/jobs`	Filter jobs by project, queue, worker, status, tag
`GET`	`/api/v1/jobs/{job_id}`	Job detail with executions and logs
`GET`	`/api/v1/jobs/dead-letter/list?project_id=...`	List DLQ jobs
`POST`	`/api/v1/jobs/{job_id}/retry`	Requeue DLQ job

Workers

Method	Path	Purpose
`POST`	`/api/v1/workers/register`	Register worker
`GET`	`/api/v1/workers`	List workers
`POST`	`/api/v1/workers/{worker_id}/heartbeat`	Record heartbeat
`POST`	`/api/v1/workers/{worker_id}/claim`	Atomically claim next job
`POST`	`/api/v1/workers/{worker_id}/jobs/{job_id}/start`	Mark job running
`POST`	`/api/v1/workers/{worker_id}/jobs/{job_id}/complete`	Complete job
`POST`	`/api/v1/workers/{worker_id}/jobs/{job_id}/fail`	Fail job and retry/DLQ

Metrics

Method	Path	Purpose
`GET`	`/api/v1/metrics/overview`	Dashboard overview metrics

Design Decisions

Backend First, Feature Count Second

The assignment prioritizes engineering quality. PulseQueue therefore implements reliability-critical backend paths before bonus features. This makes the submission stronger because the core scheduler story can be defended in code and tests.

Database as Source of Truth

PostgreSQL is the primary consistency layer. Redis is included for future distributed locks, rate limiting, and cache work, but the current claim path intentionally relies on database transactions so job ownership survives process restarts.

Atomic Claiming Strategy

Workers never claim jobs by reading and then blindly updating. The service filters eligible jobs and transitions status with a guarded write. PostgreSQL deployments can use `SKIP LOCKED`; SQLite tests use a guarded update to prove duplicate claims are prevented.

Retry Policies Are Data

Retry behavior lives in `retry_policies`, not hard-coded worker logic. Jobs can inherit from queue policy or override policy/max attempts. This supports fixed, linear, and exponential backoff.

Execution History Is Separate From Job State

`jobs` stores current state. `job_executions` and `job_logs` store history. This keeps dashboard queries simple while preserving auditability.

Worker Control Plane

Worker routes are intentionally separate from user routes. In a production version, they should use a service token or mTLS. The assignment version keeps them simple and visible so the lifecycle is easy to test.

Dashboard Shape

The dashboard is an operations console, not a landing page. It surfaces backlog, failures, workers, throughput, queue controls, and job explorer state immediately.

User Manual

1. Register and Login

Use `/api/v1/auth/register` to create a user and organization. Use `/api/v1/auth/login` to receive a bearer token. Include the token in dashboard/API calls:

```
Authorization: Bearer <token>
```

2. Create Project

Create a project under an organization. Projects isolate queues, jobs, retry policies, members, and metrics.

3. Configure Retry Policy

Create one or more retry policies:

- `fixed`: same delay between attempts.
- `linear`: delay grows by attempt number.
- `exponential`: delay doubles each attempt up to a maximum.

4. Create Queue

Queues define independent processing lanes with priority, max concurrency, retry policy, and pause/resume status.

5. Submit Jobs

Submit:

- Immediate jobs by omitting `scheduled_at`.
- Delayed jobs with `delay_seconds`.
- Scheduled jobs with `scheduled_at`.
- Recurring jobs with `cron_expression`.
- Batch jobs through `/api/v1/jobs/batch`.

Payload examples:

```
{"action": "echo", "message": "hello"}
```

```
{"action": "sleep", "seconds": 2}
```

```
{"action": "fail", "message": "simulate retry"}
```

6. Run Workers

Workers register, heartbeat, claim, start, complete, and fail jobs. The local worker runner executes demo payload actions.

7. Monitor Operations

Use the dashboard to inspect:

- Running, queued, failed, and completed jobs.
- Worker status and heartbeat freshness.
- Queue status, priority, and concurrency.
- Dead Letter Queue jobs.
- Throughput and retry rate.

8. Recover Failures

The scheduler promotes due jobs and recovers stale worker claims. Dead-letter jobs can be retried through `/api/v1/jobs/{job_id}/retry`.

Testing

Run backend tests:

```
cd backend
python -m pytest -q
```

Current critical coverage:

- Atomic claiming prevents duplicate execution.
- Jobs transition through retry and Dead Letter Queue behavior.
- Fixed, linear, and exponential retry delays.

Recommended next tests:

- API authentication and authorization.
- Queue pause/resume behavior.
- Scheduler stale-worker recovery.
- PostgreSQL `SKIP LOCKED` integration.
- Worker graceful shutdown.

Next Steps & Completed Milestones

This file outlines the implemented features and the continuation plan for advanced engineering features should you wish to expand PulseQueue further.

Completed Milestones (Rubric Highlights)

- [x] **Relational Schema Design**: Complete PostgreSQL support (Users, Organizations, Projects, Queue, Jobs, Executions, Policies, Workers, Heartbeats, Logs, DLQ entries) defined in SQLAlchemy.
- [x] **Atomic Job Claiming**: Prevent duplicate execution using guarded database status updates (compatible with SQLite) and `FOR UPDATE SKIP LOCKED` (for PostgreSQL).
- [x] **Worker Graceful Shutdown**: Added Unix/Windows signal handling (`SIGINT`, `SIGTERM`). Workers transition to `draining` state, finish active tasks, and update database status to `offline` on exit.
- [x] **Full-Featured Operations Console**:
- [x] JWT Authentication UI (Login, Signup, and Demo account bypass).
- [x] Active Project & Organization selector.
- [x] Queue Management (create queues, bind retry policies, and pause/resume triggers).
- [x] Job Explorer (status filtering, tag search, and pagination).
- [x] Manual Job Submitter (custom delay, scheduled time, cron, and preset payloads).
- [x] Job Detail Drawer (pretty-printed JSON payload, execution attempts timeline, and audit logs).
- [x] Dead Letter Queue & Retry Management (inspect failures, requeue/retry jobs).
- [x] Worker Monitor (track hostname, queues, active jobs, and status).
- [x] Policies Manager (list and create fixed, linear, and exponential retry strategies).
- [x] **Database Seeder**: Added `scripts/seed_demo.py` command for instant sandbox data.
- [x] **Testing**: Automated tests covering concurrency claim locks, backoffs, and dead letter transitions.

Continuation Plan: Advanced Work

If you have additional time and want to make the submission even more competitive, consider implementing these advanced features:

1. Concurrency & Timeout Enforcement

- **Worker Job Timeout**:
- Enforce timeouts on the worker runner. Currently, if a task hangs indefinitely, the worker thread will block.
- *How to implement*: Wrap the payload execution in a future with a timeout (e.g. `executor.submit(execute_payload).result(timeout=job.timeout_seconds)`).
- **Distributed Locking via Redis**:
- Add Redis-based distributed locking (using Redlock) to queues or specific job executions to guarantee single-concurrency across multiple distributed worker clusters.

2. Workflow Dependencies (DAGs)

- **Job Chain Execution**:
- Allow a job to declare dependency on one or more parent jobs completing successfully before it runs.
- *How to implement*:
  1. Create a `job_dependencies` table in the database mapping `parent_job_id` and `child_job_id`.
  2. Add a `BLOCKED` status to `JobStatus` enums.
  3. Modify `claim_next_job` to skip blocked jobs, and let `complete_job` check and unblock dependent child jobs once the parent job completes.

3. OpenTelemetry & Metrics Polish

- **Telemetry Integration**:
- Integrate OpenTelemetry SDK in FastAPI, worker runner, and scheduler to record trace histories of background processes.
- **Prometheus Scraper**:
- Expose `/metrics` endpoint on the FastAPI backend for Prometheus to scrape queue depths, processing times, and failure rates.

4. WebSocket Live Updates

- **Push Telemetry**:
- Replace the current HTTP short-polling on the dashboard with a WebSocket route (`/api/v1/metrics/ws`) that streams queue and worker updates in real-time.

Verification Checklist Before Submission

1. **Verify Backend Tests**: Ensure all automated tests pass: `bash cd backend python -m pytest -q`
2. **Build and Start Container Stack**: Verify that Docker Compose builds and starts all services (PostgreSQL, Redis, API, Worker, Scheduler, and React frontend): `bash docker compose up --build`
3. **Capture UI Screenshots**: Once the stack is running, log in using the demo account and capture screenshots of:
4. Dashboard Overview.
5. Queue configuration tables.
6. Jobs Explorer with the details drawer open.
7. Dead Letter Queue actions page. Save these screenshots in `docs/screenshots/` to present a clean package for evaluation.